Yanasse: Finding New Proofs from Deep Vision's Analogies

Part 2: Probability Theory → Set Theory

Alexandre Linhares*

April 2026

Abstract

This is Part 2 of the Yanasse series, applying cross-area tactic transfer to the pair Probability → SetTheory. The system identifies tactic schemas that are heavily used in probability theory but rare or absent in set theory (cardinal and ordinal arithmetic), matches source and target proof states via GPU-accelerated relational analogy on a MacBook Air (Apple MPS backend), and asks an AI reasoning agent to semantically adapt the source pattern.

The method produces 6 Lean-verified new proofs from 4 successful schema transfers out of 10 attempts (40% schema success rate). The proofs target two theorems in cardinal and ordinal arithmetic: `mul_eq_left_iff` (when cardinal multiplication absorbs: $a \cdot b = a$) and `omega0_lt_preOmega_iff` (characterizing $\omega < \text{preOmega}(x) \Leftrightarrow \omega < x$).

The deepest transfer is a *proof-architecture transfer*: the probability-theory pattern "case-split on a structural niceness predicate" (e.g., `IsSFiniteKernel` κ) maps to the set-theory pattern "case-split on whether a cardinal is infinite" ($\aleph_0 \leq a$). The two predicates share no identifiers, types, or namespaces; the connection is purely semantic. Cross-pair comparison with Part 1 (Representation Theory) confirms the (head, modifier) decomposition finding: the same three schemas that transferred to representation theory—`ext1`, `any_goals`, `by_cases`—also transfer to set theory, while the same three—`lift`, `measurability`, `congrm`—fail in both areas.

*alexandre@linhares.ltd. Affiliations: Getulio Vargas Foundation and ARGO LABS

Contents

1 Introduction **3**
1.1 Overview of the Deep Vision framework . . . 3
1.2 Results preview . . . 3

2 Methodology **4**
2.1 Pipeline summary . . . 4
2.2 Pair-specific parameters . . . 4

3 Results: Probability → Set Theory **4**
3.1 New proof P1a: `ext1`→ `Cardinal.inductionOn + mk_congr` . . . 5
3.2 New proof P1b: `any_goals`→ `any_goals lia` . . . 6
3.3 New proof P1c: `by_cases`→ top-level infinite/finite dichotomy . . . 6
3.4 New proof P2: `push`→ ordinal normalization through an embedding . . . 7

4 Cross-Pair Comparison with Part 1 **9**

5 The (Head, Modifier) Decomposition: Updated Evidence **9**

6 Failures and Their Diagnoses **10**

7 Discussion **10**
7.1 The higher proof yield . . . 10
7.2 Convergence to a single target . . . 11
7.3 Depth of transfer . . . 11
7.4 QAP entity-ratio degeneration . . . 11
7.5 Cost . . . 11

8 Conclusion **11**

A Lean Code: New Proofs **14**
A.1 Item 13: `mul_eq_left_iff`via ext1-analog (Cardinal.inductionOn + mk_congr) . . . 14
A.2 Item 15: `mul_eq_left_iff`via `any_goals lia` . . . 16
A.3 Item 17: `mul_eq_left_iff`via top-level `by_cases ha :` $\aleph_0 \leq$ `a` . . . 17
A.4 Item 19: `omega0_lt_preOmega_iff`via push-normalization (3 variants) . . . 19

1 Introduction

Part 1 of this series [1] demonstrated that proof strategy patterns can be systematically transferred across distant areas of Mathlib, producing Lean-verified new proofs at a 40% success rate on the pair Probability → Representation Theory. The key finding was the *(head, modifier) decomposition*: tactic heads are domain-gated, but modifiers carry domain-general patterns that transfer.

This paper applies the same method to a second pair: PROBABILITY → SETTHEORY. Set theory in Mathlib comprises cardinal arithmetic (`Mathlib.SetTheory.Cardinal`), ordinal arithmetic (`Mathlib.SetTheory.Ordinal`), and ordinal notation (`Mathlib.SetTheory.Ordinal.Notation`). The mathematical content—transfinite induction, cardinal absorption laws, ordinal fixed-point theory—is structurally distant from probability's measure-theoretic world.

The pair is ranked second overall in the Yanasse pair-potential metric (Section 2), and the target area (`Mathlib.SetTheory`) contains 2,076 extracted proof states, roughly three times the size of the Representation Theory target (672).

1.1 Overview of the Deep Vision framework

The Deep Vision framework [2,12], growing out of the Fluid Analogies Research Group tradition [3–5], represents any structured domain as a *relational network*: entities connected by typed, directed relations. An analogy optimization finds the entity permutation that maximizes relational consistency between two networks. The matching code (`deep_vision_lib.py`) is entirely domain-independent: it receives two relational structures and returns a similarity score and entity correspondence, without knowing which domain it is processing.

Chess analogies. In chess [6–11], entities are pieces and relations are attack, defense, blocking, confinement, and pinning. Two pieces in entirely different positions can play "the same role" if their relational profiles match. The matcher discovers such correspondences across positions that differ radically in surface features—12-piece endgames matched against 5-piece positions, opening traps matched against sparse endgames [1].

From chess to proofs. The same matching engine, with only the relation extractor swapped, handles Lean 4 proof states: entities become hypotheses, goals, and types; relations become rewrite, fit/apply, head-match, structure, equality, reflexive, witness, bidirectional, kernel/simp, decidable, and lemma-needed (14 relation types). No chess knowledge is used; no proof-specific heuristics are added to the matcher. The proof-state matcher has previously been used to discover a formal proof of Wolstenholme's theorem [13] in Lean 4 (see also [14]).

1.2 Results preview

Applying the method to Probability → Set Theory produces:

- 6 Lean-verified new proofs from 4 successful schema transfers out of 10 attempts (40% schema success rate, with one schema producing 3 independent proof variants).
- A *proof-architecture transfer*: the deepest result reorganizes a cardinal-arithmetic proof around a top-level infinite/finite dichotomy, mirroring probability's s-finite/non-s-finite split (Section 3.3).
- Cross-pair comparison confirming the (head, modifier) decomposition from Part 1 (Section 4).

- 6 negative results with mathematical diagnoses (Section 6).

2 Methodology

The methodology is identical to Part 1 [1]. We summarize the pipeline and note the parameters specific to this pair.

2.1 Pipeline summary

1. **Schema extraction.** Parse the Mathlib proof-state corpus (217,133 entries). Each tactic invocation becomes a schema tuple (*head*, *arity*, *has_with*, *uses_lemma*).

2. **z-score ranking.** For each (area, schema) pair, compute the z-score of the schema's frequency across 27 areas. A schema with $z \geq +2$ in the source and $z \leq -1$ (or absent) in the target is a transfer candidate.

3. **GPU-accelerated analogy matching.** Sample 8 source proofs per schema, match each against all 2,076 Set Theory proof states using the `BatchMatcher` on MPS. Retain the top-5 target analogues by normalized score.

4. **Semantic adaptation.** A fresh AI reasoning session (Claude Opus, 50-minute budget) adapts the source tactic's *mathematical strategy*—not its syntax—to the target theorem.

5. **Lean verification.** The adapted proof is compiled with `lake env lean`. Exit code 0 with no `sorry` constitutes verification.

2.2 Pair-specific parameters

- **Source:** `Mathlib.Probability` (30,384 tactic invocations, the largest Mathlib area).
- **Target:** `Mathlib.SetTheory` (2,076 proof states—3× larger than the Representation Theory target).
- **Matching cost:** $8 \times 2{,}076 = 16{,}608$ individual analogy computations per schema, completing in approximately 15–60 minutes on MPS depending on source proof-state size.
- **Top-10 transfer candidates** (by gap score): `filter_upwards`, `congr with`, `fun_prop`, `ext1`, `lift`, `any_goals`, `measurability`, `by_cases`, `congrm`, `push`.

Note that the same 10 schemas appear as in Part 1 (with the addition of `push`, replacing `all_goals` which was already tested against Representation Theory). This overlap enables direct cross-pair comparison.

3 Results: Probability → Set Theory

Four transfer attempts succeeded, producing 6 Lean-verified new proofs (item 19 yielded 3 independent proof variants). The target theorems are:

Table 1: Transfer results for Probability → Set Theory.

Item	Schema head	with	Ar.	Result	Adaptation
10	`filter_upwards`	✓	1	Failed	No `Filter` in ordinal arith.
11	`congr`	✓	0	Failed	No extensionality for Cardinal
12	`fun_prop`		0	Failed	Domain-gated (Measurable)
13	`ext1`		1	**New proof**	`inductionOn + mk_congr`
14	`lift`	✓	5	Failed	No `CanLift` for Ordinal
15	`any_goals`		1	**New proof**	`any_goals lia`
16	`measurability`		0	Failed	Domain-gated (σ-alg.)
17	`by_cases`		4	**New proof**	`by_cases ha : ℵ₀ ≤ a`
18	`congrm`		5	Failed	No shared outer operators
19	`push`		3	**New proofs (3)**	`rwa [← preOmega_omega0]`

P1. Cardinal multiplication absorption (`mul_eq_left_iff`, items 13, 15, 17). For cardinals a, b:

$$a \cdot b = a \iff \big(\max(\aleph_0, b) \le a \ \wedge\ b \ne 0\big) \ \vee\ b = 1 \ \vee\ a = 0.$$

Three new proofs via three different transferred schemas.

P2. Pre-omega comparison (`omega0_lt_preOmega_iff`, item 19). For ordinals:

$$\omega < \mathrm{preOmega}(x) \iff \omega < x.$$

Three proof variants via the transferred "push" normalization pattern.

3.1 New proof P1a: `ext1` → `Cardinal.inductionOn + mk_congr`

Theorem. For cardinals a and b, the absorption law $a \cdot b = a$ holds if and only if $\max(\aleph_0, b) \le a$ and $b \ne 0$, or $b = 1$, or $a = 0$.

Context. Cardinal arithmetic diverges sharply from natural-number arithmetic once infinite cardinals appear. The absorption law $a \cdot b = a$ when a is infinite and $b \le a$ was first established (for $a \cdot a = a$) by Hessenberg in 1906 [17], later generalized by Tarski's theorem on cardinal products [18] and systematized in Jech [15] (Chapter 5) and Kunen [16] (Chapter I, §10). Holz, Steffens, and Weitz [19] provide a modern treatment in the context of Shelah's pcf theory [20], which reveals deeper structure behind singular cardinal arithmetic. The characterization in `mul_eq_left_iff`—providing the complete equivalence including the finite and degenerate cases—appears in `Mathlib.SetTheory.Cardinal.Arithmetic`, whose formalization follows Jech's treatment. The result is a workhorse lemma for cardinal exponentiation and Galvin–Hajnal-type bounds [21].

New proof (ext1 analog). The original proof dispatches the two "easy" backward cases ($b = 1$ and $a = 0$) with `all_goals simp`, which invokes the simplification lemmas `mul_one` and `zero_mul`. The transferred proof replaces this automation with an *explicit type-equivalence witness*, the set-theoretic analog of the extensionality principle:

- For $a \cdot 1 = a$: lift a to a concrete type α via `Cardinal.inductionOn`; express $\#\alpha \cdot 1$ as $\#(\alpha \times \texttt{PUnit})$ via `Cardinal.mul_def`; close with `Cardinal.mk_congr (Equiv.prodPUnit α)`, the canonical equivalence $\alpha \times \texttt{PUnit} \simeq \alpha$.
- For $0 \cdot b = 0$: analogously, lift b and use `Equiv.pemptyProd β` for $\texttt{PEmpty} \times \beta \simeq \texttt{PEmpty}$.

Source pattern. In probability theory, `ext1` ω reduces set equality $S = T$ to pointwise membership $\forall \omega,\ \omega \in S \leftrightarrow \omega \in T$—a workhorse for measurability proofs [1].

Adaptation. Direct `ext1` fails: Cardinals have no `@[ext]` lemma (they are quotient types, not structures with component-wise decomposition). The semantic analog of "reduce equality to constituent-level witness" is: lift to a concrete representative type via `inductionOn`, then exhibit an explicit type equivalence via `mk_congr`.

Analogy matching. The source proof (`measurable_stoppedValue`) had 51 entities; the top-3 target analogues were all tied at QAP score 18.19, matched from `Cardinal/Arithmetic.lean`, `Cardinal/Basic.lean`, and `Ordinal/FixedPoint.lean`.

3.2 New proof P1b: `any_goals` → `any_goals lia`

Theorem. Same as P1a: `mul_eq_left_iff`.

New proof. The transferred tactic `any_goals lia` replaces the original `all_goals simp` at the cleanup step of the backward direction. After the case-split `rintro (... | rfl | rfl)`, two sub-goals remain: $a \cdot 1 = a$ (from $b = 1$) and $0 \cdot b = 0$ (from $a = 0$). The linear integer arithmetic decision procedure `lia` closes both via internal normalization on Cardinals.

Source pattern. In `Mathlib.Probability.Kernel.IonescuTulcea.Traj`, the tactic `any_goals lia` dispatches trivial arithmetic side-conditions after a case split on trajectory indices. It is a *partial-sweep combinator*: broadcast a decision procedure across all sub-goals, close the easy ones, leave the hard ones for subsequent tactics.

Adaptation. The transfer is *literal*: the source tactic `any_goals lia` compiles directly in the target context, closing the same kind of arithmetic identities ($a \cdot 1 = a$, $0 \cdot b = 0$) that its probability-theory counterpart closes (index arithmetic like $i \leq k + 1$).

Contrast with Part 1. In Part 1, `any_goals rfl` transferred to Representation Theory for the *same architectural reason*: after a case-split, one branch is definitionally trivial. The inner tactic differs (`rfl` vs. `lia`), but the proof pattern—"broadcast a simple closer across heterogeneous sub-goals"—is identical.

3.3 New proof P1c: `by_cases` → top-level infinite/finite dichotomy

Theorem. Same as P1a: `mul_eq_left_iff`.

Context. This is the deepest semantic transfer in the paper. The original Mathlib proof of `mul_eq_left_iff` splits the biconditional first (`refine ⟨fun h ⇒ ?, ?⟩`), then uses `rcases le_or_gt` $\aleph_0$ `a` deep inside the forward direction. The transferred proof reorganizes the entire argument around a top-level `by_cases` on whether a is infinite.

New proof.

1. `by_cases ha : ℵ₀ ≤ a` — the structural "niceness split."
2. **Infinite case** ($\aleph_0 \le a$): Cardinal multiplication absorbs: $a \cdot b = \max(a, b)$ by `mul_eq_max_of_aleph0_le_left`. From $a \cdot b = a$ derive $b \le a$ via the max characterization. The backward direction dispatches the three disjuncts with `mul_eq_left`, `simp`, `simp`.
3. **Finite case** ($a < \aleph_0$): The first disjunct (requiring $\aleph_0 \le a$) is impossible. A nested `by_cases h2a : a = 0` handles the sub-degenerate case. When $a \neq 0$, show b must also be finite (otherwise $a \cdot b \ge b \ge \aleph_0 > a$), cast to $\mathbb{N}$ via `lt_aleph0`, and use `Nat.mul_eq_left`.

Source pattern. In `Mathlib.Probability.Kernel.Composition.MeasureComp`, the theorem `comp_compProd_comm` uses:

```
by_cases hκ : IsSFiniteKernel κ
```

This is a *structural niceness guard*: s-finite kernels support composition-product theory; non-s-finite kernels make `compProd` collapse to zero, so `simp` with collapsing lemmas closes the negative case in one line.

Adaptation. The semantic mapping is:

	Source (Probability)	Target (Set Theory)
Niceness predicate	`IsSFiniteKernel` κ	$\aleph_0 \le a$
What it enables	Composition-product theory	Max-absorption arithmetic
Negative case	`compProd` collapses → `simp`	Falls to $\mathbb{N}$ arithmetic
Positive case	Extensionality + rewrite	`mul_eq_max` + `max_eq_left`

The predicates share no identifiers, types, or namespaces. The connection is semantic: both determine whether the main algebraic theory of their respective domains applies. This is a transfer at the level of *proof architecture*, not tactic syntax.

Cross-pair comparison. In Part 1, `by_cases` transferred to Representation Theory as "case-split on $f = 0$" (a degenerate-morphism guard). The Set Theory adaptation is arguably deeper: the probability-theory analog (`IsSFiniteKernel`) and the set-theory analog ($\aleph_0 \le a$) both partition their domain's objects into a "nice regime" where the main algebraic theory works and a "degenerate regime" requiring qualitatively different reasoning. The morphism-triviality split in Part 1 was mathematically unnecessary (both branches closed identically); the infinite/finite split here is essential.

3.4 New proof P2: `push` → ordinal normalization through an embedding

Theorem. For ordinals x:

$$\omega < \mathrm{preOmega}(x) \iff \omega < x.$$

Here preOmega : Ord ↪ Ord is the order embedding enumerating initial ordinals, and ω is the first infinite ordinal.

Context. The function preOmega enumerates the *initial ordinals*—the smallest ordinals of each cardinality—a construction going back to Hartogs [22] and systematized by von Neumann [23]. In modern notation, $\text{preOmega}(\alpha) = \omega_\alpha$ for infinite α, with the convention $\text{preOmega}(n) = n$ for finite n. The fixed-point property $\text{preOmega}(\omega) = \omega$ reflects the fact that ω is the ω-th initial ordinal—the smallest aleph fixed point, studied extensively in Jech [15] (Chapter 2) and Kunen [16] (Chapter I, §7). Drake [24] gives a thorough treatment of the aleph hierarchy and its fixed-point structure. The biconditional `omega0_lt_preOmega_iff` exploits the fixed-point property and the strict monotonicity of preOmega as an order embedding; it appears in `Mathlib.SetTheory.Cardinal.Aleph`. The original Mathlib proof uses a targeted rewrite: `conv_lhs => rw [← preOmega_omega0, preOmega_lt_preOmega]`.

New proofs (3 variants). The transferred "push" normalization pattern—normalize by pushing a structural wrapper through an operator—yields three proof variants:

1. **Constructor + rwa.** Split the iff; in each direction, `rwa [← preOmega_omega0, preOmega_lt_preOmega]` introduces $\text{preOmega}(\omega)$ and cancels the paired embeddings. Each rewrite is applied exactly once, avoiding the simp loop (see below).
2. **Targeted rewrite.** `nth_rw 1 [show` $\omega = \text{preOmega}(\omega)$ `from preOmega_omega0.symm]; exact preOmega_lt_preOmega`. Replace only the first ω, then close with the embedding's monotonicity lemma.
3. **Calc chain.** A two-step calculation making the "push-through" steps explicit: $\omega < \text{preOmega}(x) \Leftrightarrow \text{preOmega}(\omega) < \text{preOmega}(x) \Leftrightarrow \omega < x$.

Source pattern. In `Mathlib.Probability.Process.Stopping`, the tactic `push _ ∈ _` normalizes set membership by pushing $\in$ through set-builder notation, preimages, and indexed unions: $\omega \in \{x \mid P\,x\} \rightsquigarrow P\,\omega$.

Adaptation. The source pushes $\in$ through syntactic set-construction wrappers; the target pushes $<$ through preOmega, an order-embedding wrapper. The key lemmas play parallel roles:

Source (Probability)	Target (Set Theory)
`Set.mem_setOf_eq`	`preOmega_lt_preOmega`
(unfold set-builder)	(cancel paired embeddings)
`Set.mem_preimage`	`preOmega_omega0`
(push through preimage)	(introduce embedding at fixed point)

A failed approach. The most natural "push" analog— `simp only [← preOmega_omega0, preOmega_lt_preOmega]`—loops: the backward rewrite $\omega \to \text{preOmega}(\omega)$ and the forward simplification that cancels paired embeddings create an infinite cycle. The source's `push _ ∈ _` never loops because set-builder unfolding is one-directional. The `rwa`-based adaptation avoids this by applying each rewrite exactly once.

Table 2: Schema transfer results across two target areas.

Schema head	RepTheory (Part 1)	SetTheory (Part 2)	Category
`filter_upwards`	**New proof**	Failed	Domain-gated
`congr with`	**New proof**	Failed	Domain-general
`fun_prop`	Failed	Failed	Domain-gated
`ext1`	Failed	**New proof**	Domain-general
`lift`	Failed	Failed	Domain-gated
`any_goals`	**New proof**	**New proof**	Domain-general
`measurability`	Failed	Failed	Domain-gated
`by_cases`	**New proofs (2)**	**New proof**	Domain-general
`congrm`	Failed	Failed	Domain-gated
`push`	—	**New proofs (3)**	Domain-general

4 Cross-Pair Comparison with Part 1

Seven of the ten schemas tested against Set Theory were also tested against Representation Theory in Part 1. This enables direct comparison:

Finding 1 (Consistent transfer/failure across targets)**.** Of the 7 schemas tested in both areas:

- **2 transfer in both:** `any_goals`, `by_cases`. These are domain-general combinators whose proof-architectural patterns (partial sweep, niceness guard) exist in every mathematical area.
- **3 fail in both:** `fun_prop`, `lift`, `measurability`. These are domain-gated tactics requiring measure-theoretic or topological type structures absent from both targets.
- **2 differ:** `filter_upwards` (succeeds in RepTheory via modifier transfer, fails in SetTheory where ordinal arithmetic has no Filter API), `ext1` (fails in RepTheory where categorical extensionality is structurally incompatible, succeeds in SetTheory where `Cardinal.inductionOn` provides a semantic analog).

The consistency of the "never-transfer" group—`fun_prop`, `lift`, `measurability`—across both targets strengthens the evidence that these schemas are *intrinsically* domain-gated, not merely unlucky in their target matching. Future runs could skip them entirely for non-measure-theoretic targets, saving ∼30% of compute.

The two schemas that differ between targets (`filter_upwards` and `ext1`) illustrate that the (head, modifier) decomposition is target-sensitive: the modifier of `filter_upwards` ("`[LIST] with` ω") finds a structural analog in representation theory's extensionality-based reasoning but not in ordinal arithmetic's algebraic reasoning, while `ext1`'s principle ("reduce equality to constituent-level witness") finds a semantic analog in cardinal arithmetic's type-equivalence infrastructure but not in categorical extensionality.

5 The (Head, Modifier) Decomposition: Updated Evidence

Part 1 introduced the (head, modifier) decomposition. The Set Theory results provide additional evidence and one refinement.

Finding 2 (Proof-architecture transfer)**.** The deepest transfers are not at the tactic level but at the *proof-architecture* level: the decision to organize a proof around a top-level case-split on a structural

niceness predicate (`by_cases`, item 17) transfers across areas because the *architecture*—split on niceness, handle degenerate case trivially, proceed with main theory in nice case—is domain-general, even when the specific niceness predicates are domain-specific.

The `by_cases` transfer to Set Theory is qualitatively different from its Part 1 counterpart. In Part 1, the case-split ($f = 0$) was mathematically unnecessary (both branches closed identically); the transfer demonstrated that the *pattern* works but not that it *improves* the proof. In Part 2, the infinite/finite split is mathematically essential: the infinite case uses max-absorption ($a \cdot b = \max(a, b)$) while the finite case casts to $\mathbb{N}$ and uses `Nat.mul_eq_left`. These are fundamentally different proof strategies, and the transferred architecture makes this dichotomy explicit at the top level rather than burying it inside the forward direction.

6 Failures and Their Diagnoses

Item 10: `filter_upwards.` `filter_upwards` requires `Filter.Eventually` goals. Set Theory's ordinal notation proofs (the top-1 target: `nf_repr_split'`) have goals of the form $\texttt{NF}\, o \wedge \texttt{repr}\, o = \omega \cdot \texttt{repr}\, o' + m$—propositional conjunctions about ordinal notation, not filter membership. The 55-to-3 entity ratio (source to target) also produced a spurious QAP score.

Item 11: `congr with.` `congr with` requires an extensionality theorem for the goal type. `Cardinal` has no `@[ext]` lemma (confirmed by Lean: "No applicable extensionality theorem found for type Cardinal"). Positive controls confirmed that `congr with n` *does* work on ordinal function equality (`nfp_add_zero`) and `Cardinal.sum`—a genuine transfer exists at rank 3, but the QAP matcher could not discriminate it from the degenerate rank-1 mapping (37-to-3 entity ratio).

Item 12: `fun_prop.` `fun_prop` handles `Measurable`, `Continuous`, and `Differentiable` goals. Set Theory has zero such goal types. Same root cause as Part 1 item 2.

Item 14: `lift.` `lift` requires `CanLift` instances (e.g., $\mathbb{R}^{\infty}_{\geq 0} \to \mathbb{R}_{\geq 0}$). No `CanLift` instance exists for `Ordinal` in Mathlib: ordinals have no canonical simpler base type. Five Lean probes all failed at typeclass synthesis. Same root cause as Part 1 item 5.

Item 16: `measurability.` A σ-algebra predicate closure tactic. The target theorem (`repr_opow`) proves that ordinal notation representation preserves exponentiation—a purely algebraic statement with no measurability predicates. Same root cause as Part 1 item 7.

Item 18: `congrm.` `congrm` requires both sides of an equality to share a common outer-operator skeleton (integrals, products, set membership). The target goal $\omega < \text{preOmega}(x) \Leftrightarrow \omega < x$ is an atomic order comparison—no composite structure to decompose. Five QAP-matched targets were identically scored (23.53), indicating no discriminative structural match. Same root cause as Part 1 item 9.

7 Discussion

7.1 The higher proof yield

Set Theory's schema success rate ($4/10 = 40\%$) matches Representation Theory's, but the *proof yield* is higher: 6 Lean-verified proofs versus 4. This may reflect the target area's richer library of arithmetic lemmas. Three of the four successful transfers target `mul_eq_left_iff`, a theorem with a complex case structure and multiple algebraic regimes. Such theorems offer more "surface area"

for diverse proof strategies. In contrast, Representation Theory's successful transfers targeted two different theorems, each with a simpler proof structure.

7.2 Convergence to a single target

Three of the four successful schemas (items 13, 15, 17) converge on the same target theorem (`mul_eq_left_iff`). This is partly an artifact of the QAP matcher—the 2,076-entry target pool has a small number of high-scoring "attractor" entries—and partly reflects that `mul_eq_left_iff` is genuinely a rich theorem with a case-analytic structure amenable to multiple proof strategies.

7.3 Depth of transfer

The transfers range from shallow (item 15: literal `any_goals lia` compiles directly) to deep (item 17: the "niceness guard" architecture requires semantic reasoning about the role of $\aleph_0 \leq a$ in cardinal arithmetic). Item 19 is intermediate: the "push normalization" pattern is genuinely structural, but the target theorem is already a simple rewrite.

7.4 QAP entity-ratio degeneration

Items 10 and 11 both suffered from degenerate QAP scores caused by extreme entity-ratio mismatches (55-to-3, 37-to-3). When the source proof state has 10× more entities than the target, the matcher can achieve high scores by mapping a small cluster of source entities to the target while ignoring the rest. Item 11's positive control (a genuine transfer at rank 3) was masked by the degenerate rank-1 score.

This motivates a planned improvement: penalizing matches where the entity ratio exceeds a threshold (e.g., $5:1$), so that the matcher favors structurally comparable proof states.

7.5 Cost

The full pipeline for this pair consumed:

- ∼25 minutes of GPU time per schema (MPS) for analogy matching ($8 \times 2{,}076$ comparisons).
- ∼8 hours of AI reasoning across 10 semantic adaptation sessions.
- ∼$100–150 in API cost.

8 Conclusion

The Probability → Set Theory pair achieves a 40% schema success rate (4/10), producing 6 new Lean-verified proofs. The key result is a *proof-architecture transfer*: the probability-theory pattern "case-split on a structural niceness predicate" maps to cardinal arithmetic's infinite/finite dichotomy, demonstrating that cross-area transfer can operate at the level of proof organization, not just tactic invocation.

Cross-pair comparison with Part 1 confirms that domain-gated schemas (`fun_prop`, `lift`, `measurability`) consistently fail across targets, while domain-general combinators (`any_goals`, `by_cases`) consistently transfer. This consistency enables predictive filtering: future runs can skip known-doomed schemas for non-measure-theoretic targets.

Part 3 will apply the method to `Probability` → `Condensed`, the hardest target tested so far (condensed mathematics eliminates the Filter and σ-algebra vocabulary that probability-theory schemas rely on).

Acknowledgments

Named after Horacio Hideki Yanasse. The Deep Vision matching engine was originally developed for chess cognition research; its application to formal mathematics was enabled by the domain-independent design of `deep_vision_lib.py`. All computations were performed on a MacBook Air (Apple M-series, 32 GB unified memory) using PyTorch 2.8 on the MPS backend.

References

[1] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 1: Probability Theory → Representation Theory," ARGO LABS Report for Distribution, April 2026.

[2] A. Linhares, "Deep Vision," ARGO LABS Internal Report, 2026.

[3] D. R. Hofstadter and the Fluid Analogies Research Group, *Fluid Concepts & Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. New York: Basic Books, 1995.

[4] D. R. Hofstadter, *Gödel, Escher, Bach: An Eternal Golden Braid*, 20th anniversary ed. New York: Basic Books, 1999.

[5] D. R. Hofstadter and E. Sander, *Surfaces and Essences: Analogy as the Fuel and Fire of Thinking*. New York: Basic Books, 2013.

[6] A. Linhares, "An active symbols theory of chess intuition," *Minds and Machines*, vol. 15, pp. 131–151, 2005.

[7] A. Linhares and P. Brum, "Understanding our understanding of strategic scenarios: What role do chunks play?" *Cognitive Science*, vol. 31, no. 6, pp. 989–1007, 2007.

[8] A. Linhares and A. E. T. A. Freitas, "Questioning Chase and Simon's (1973) 'Perception in Chess'," *New Ideas in Psychology*, vol. 28, no. 1, pp. 64–78, 2010.

[9] A. Linhares, A. E. T. A. Freitas, A. Mendes, and J. S. Silva, "Entanglement of perception and reasoning in the combinatorial game of chess," *Cognitive Systems Research*, vol. 13, no. 1, pp. 72–86, 2012.

[10] A. Linhares and D. M. Chada, "What is the nature of the mind's pattern-recognition process?" *New Ideas in Psychology*, vol. 31, no. 2, pp. 108–121, 2013.

[11] A. Linhares, "The emergence of choice: Decision-making and strategic thinking through analogies," *Information Sciences*, vol. 259, pp. 36–56, 2014.

[12] A. Linhares, "Deep Vision: Seeing the essence of proofs through analogy," ARGO LABS Internal Report, 2026.

[13] A. Linhares, "Deep Vision: A formal proof of Wolstenholme's theorem in Lean 4," ARGO LABS Report for Distribution, 2026.

[14] A. Linhares, "A sketch of Deep Vision's study of mathematics," ARGO LABS Report for Distribution, 2026.

[15] T. Jech, *Set Theory: The Third Millennium Edition, Revised and Expanded*, Springer Monographs in Mathematics. Berlin: Springer-Verlag, 2003.

[16] K. Kunen, *Set Theory: An Introduction to Independence Proofs*, Studies in Logic, vol. 102. London: College Publications, 2011.

[17] G. Hessenberg, "Grundbegriffe der Mengenlehre," *Abhandlungen der Friesschen Schule*, N.S., vol. 1, no. 4, pp. 485–706, 1906.

[18] A. Tarski, "Quelques théorèmes sur les alephs," *Fundamenta Mathematicae*, vol. 7, pp. 1–14, 1925.

[19] M. Holz, K. Steffens, and E. Weitz, *Introduction to Cardinal Arithmetic*, Birkhäuser Advanced Texts. Basel: Birkhäuser Verlag, 1999.

[20] S. Shelah, *Cardinal Arithmetic*, Oxford Logic Guides, vol. 29. Oxford: Oxford University Press, 1994.

[21] F. Galvin and A. Hajnal, "Inequalities for cardinal powers," *Ann. of Math.*, vol. 101, no. 3, pp. 491–498, 1975.

[22] F. Hartogs, "Über das Problem der Wohlordnung," *Math. Annalen*, vol. 76, no. 4, pp. 438–443, 1915.

[23] J. von Neumann, "Zur Einführung der transfiniten Zahlen," *Acta Litterarum ac Scientiarum Regiae Universitatis Hungaricae Francisco-Josephinae, Sectio Scientiarum Mathematicarum*, vol. 1, pp. 199–208, 1923.

[24] F. R. Drake, *Set Theory: An Introduction to Large Cardinals*, Studies in Logic and the Foundations of Mathematics, vol. 76. Amsterdam: North-Holland, 1974.

A Lean Code: New Proofs

A.1 Item 13: mul_eq_left_iff via ext1-analog (Cardinal.inductionOn + mk_congr)

```
import Mathlib.SetTheory.Cardinal.Arithmetic

open Cardinal in
theorem mul_eq_left_iff' {a b : Cardinal} :
    a * b = a <-> max ℵ₀ b <= a
      ∧ b ≠ 0 ∨ b = 1 ∨ a = 0 := by
  rw [max_le_iff]
  refine <fun h => ?_, ?_>
  . rcases le_or_gt ℵ₀ a with ha | ha
    . have : a ≠ 0 := by
        rintro rfl
        exact ha.not_gt aleph0_pos
      left
      rw [and_assoc]
      use ha
      constructor
      . rw [<- not_lt]
        exact fun hb =>
          ne_of_gt (hb.trans_le (le_mul_left this)) h
      . rintro rfl
        apply this
        rw [mul_zero] at h
        exact h.symm
    right
    by_cases h2a : a = 0
    . exact Or.inr h2a
    have hb : b ≠ 0 := by
      rintro rfl
      apply h2a
      rw [mul_zero] at h
      exact h.symm
    left
    rw [<- h, mul_lt_aleph0_iff, lt_aleph0,
        lt_aleph0] at ha
    rcases ha with (rfl | rfl | <<n, rfl>, <m, rfl>>)
    . contradiction
    . contradiction
    rw [<- Ne] at h2a
    rw [<- Cardinal.one_le_iff_ne_zero] at h2a hb
    norm_cast at h2a hb h ⊢
    apply le_antisymm _ hb
    rw [<- not_lt]
    apply fun h2b => ne_of_gt _ h
    conv_rhs => left; rw [<- mul_one n]
    rw [Nat.mul_lt_mul_left]
    . exact id
    apply Nat.lt_of_succ_le h2a
  . rintro (<<ha, hab>, hb> | rfl | rfl)
    . rw [mul_eq_max_of_aleph0_le_left ha hb,
```

```
        max_eq_left hab]
  -- ext1 analog: reduce cardinal equality to
  -- type equivalence via inductionOn + mk_congr
  . induction a using Cardinal.inductionOn with
    | mk α =>
      rw [show (1 : Cardinal) =
          Cardinal.mk PUnit from
            (Cardinal.mk_eq_one PUnit).symm]
      rw [Cardinal.mul_def]
      exact Cardinal.mk_congr
        (Equiv.prodPUnit α)
  . induction b using Cardinal.inductionOn with
    | mk β =>
      rw [show (0 : Cardinal) =
          Cardinal.mk PEmpty from
            (Cardinal.mk_eq_zero PEmpty).symm]
      rw [Cardinal.mul_def]
      exact Cardinal.mk_congr
        (Equiv.pemptyProd β)
```

A.2 Item 15: mul_eq_left_iff via any_goals lia

```
import Mathlib.SetTheory.Cardinal.Arithmetic

open Cardinal

theorem mul_eq_left_iff_reproof {a b : Cardinal} :
    a * b = a <-> max ℵ₀ b <= a
      ∧ b ≠ 0 ∨ b = 1 ∨ a = 0 := by
  rw [max_le_iff]
  refine <fun h => ?_, ?_>
  -- [forward direction: identical to Mathlib]
  . rcases le_or_gt ℵ₀ a with ha | ha
    . have : a ≠ 0 := by
        rintro rfl; exact ha.not_gt aleph0_pos
      left; rw [and_assoc]; use ha
      constructor
      . rw [<- not_lt]
        exact fun hb =>
          ne_of_gt (hb.trans_le (le_mul_left this)) h
      . rintro rfl; apply this
        rw [mul_zero] at h; exact h.symm
    right
    by_cases h2a : a = 0
    . exact Or.inr h2a
    have hb : b ≠ 0 := by
      rintro rfl; apply h2a
      rw [mul_zero] at h; exact h.symm
    left
    rw [<- h, mul_lt_aleph0_iff, lt_aleph0,
        lt_aleph0] at ha
    rcases ha with (rfl | rfl | <<n, rfl>, <m, rfl>>)
    . contradiction
    . contradiction
    rw [<- Ne] at h2a
    rw [<- Cardinal.one_le_iff_ne_zero] at h2a hb
    norm_cast at h2a hb h ⊢
    apply le_antisymm _ hb
    rw [<- not_lt]
    apply fun h2b => ne_of_gt _ h
    conv_rhs => left; rw [<- mul_one n]
    rw [Nat.mul_lt_mul_left]
    . exact id
    apply Nat.lt_of_succ_le h2a
  . rintro (<<ha, hab>, hb> | rfl | rfl)
    . rw [mul_eq_max_of_aleph0_le_left ha hb,
          max_eq_left hab]
    -- TRANSFERRED: any_goals lia (from Probability)
    -- Original: all_goals simp
    any_goals lia
```

A.3 Item 17: mul_eq_left_iff via top-level by_cases ha : $\aleph_0 \leq$ a

```
import Mathlib.SetTheory.Cardinal.Arithmetic

open Cardinal

theorem mul_eq_left_iff_reproof {a b : Cardinal} :
    a * b = a <-> max ℵ₀ b <= a
      ∧ b ≠ 0 ∨ b = 1 ∨ a = 0 := by
  -- TOP-LEVEL by_cases: structural niceness split
  by_cases ha : ℵ₀ <= a
  . -- INFINITE CASE: multiplication = max
    rw [show max ℵ₀ b <= a <->
        ℵ₀ <= a ∧ b <= a from max_le_iff]
    constructor
    . intro h
      have ha0 : a ≠ 0 :=
        ne_of_gt (aleph0_pos.trans_le ha)
      have hb0 : b ≠ 0 := by
        rintro rfl; rw [mul_zero] at h
        exact ha0 h.symm
      left
      refine <<ha, ?_>, hb0>
      have h1 :=
        mul_eq_max_of_aleph0_le_left ha hb0
      rw [h] at h1
      exact (max_le_iff.mp (le_of_eq h1.symm)).2
    . rintro (<<_, hab>, hb> | rfl | rfl)
      . exact mul_eq_left ha hab hb
      . simp
      . simp
  . -- FINITE CASE: reduces to Nat arithmetic
    push Not at ha
    constructor
    . intro h
      right
      by_cases h2a : a = 0
      . exact Or.inr h2a
      . left
        have hb : b < ℵ₀ := by
          by_contra hb'
          push Not at hb'
          have := mul_eq_max_of_aleph0_le_right
            h2a hb'
          rw [h, max_eq_right
            (ha.trans_le hb').le] at this
          exact ne_of_lt
            (ha.trans_le hb') this
        rw [lt_aleph0] at ha hb
        obtain <n, rfl> := ha
        obtain <m, rfl> := hb
        have hn : n ≠ 0 := by
          exact_mod_cast h2a
        norm_cast at h ⊢
```

```
        exact (Nat.mul_eq_left hn).mp h
    . rintro (<<h1, _>, _> | rfl | rfl)
      . exact absurd (le_of_max_le_left h1)
          (not_le.mpr ha)
      . simp
      . simp
```

A.4 Item 19: omega0_lt_preOmega_iff via push-normalization (3 variants)

```
import Mathlib.SetTheory.Cardinal.Aleph

open Ordinal

-- Variant 1: constructor + rwa
theorem omega0_lt_preOmega_iff_v1
    {x : Ordinal} :
    ω < preOmega x <-> ω < x := by
  constructor
  . intro h
    rwa [<- preOmega_omega0,
         preOmega_lt_preOmega] at h
  . intro h
    rwa [<- preOmega_omega0,
         preOmega_lt_preOmega]

-- Variant 2: targeted rewrite
theorem omega0_lt_preOmega_iff_v2
    {x : Ordinal} :
    ω < preOmega x <-> ω < x := by
  nth_rw 1 [show (ω : Ordinal) = preOmega ω
    from preOmega_omega0.symm]
  exact preOmega_lt_preOmega

-- Variant 3: calc chain
theorem omega0_lt_preOmega_iff_v3
    {x : Ordinal} :
    ω < preOmega x <-> ω < x := by
  calc ω < preOmega x
      <-> preOmega ω < preOmega x :=
        by rw [preOmega_omega0]
    _ <-> ω < x := preOmega_lt_preOmega
```